

จากไอเดียสู่ระบบจริง

ง่ายกว่าที่คิด

AI-Powered Internal Developer Framework
สนับสนุนทุกไอเดีย ให้ deploy ได้เร็ว ปลอดภัย และยั่งยืน
สิงหาคม 2026

วาระการประชุม (90 นาที)

ขั้นตอน	หัวข้อ	เวลา
1	☀️ จุดเริ่มต้น: ไอเดียที่ดีสมควรได้รับการสนับสนุน	15 นาที
2	🚀 Journey: จากไอเดียสู่ระบบจริง	20 นาที
3	🤝 ทีมของเรา: ใครช่วยอะไรได้บ้าง	15 นาที
4	📖 AI ช่วยอะไรได้บ้าง	15 นาที
5	💡 เริ่มต้นอย่างไร	15 นาที

องค์ประชุม (1/2)

Maintenance (User)

- เซอร์โอดีและความต้องการ
- ให้ feedback
- เลือก pilot projects

SA

- ออกแบบ architecture
- Wiki schema
- Approval workflow

AI-Eng

- LLM integration
- Auto-doc generation
- MCP, auto PR

Dev

- Implementation
- Workflow, templates

องค์ประชุม (2/2)

Infra&Security

- k8s, GitLab, ArgoCD
- Vault, RBAC, monitoring
- Network policies

ขั้นตอนแรก

จุดเริ่มต้น: ไอเดียที่ดีสมควรได้รับการสนับสนุน

เวลา: 15 นาที | ผู้นำเสนอ: Maintenance (User)

อะไรที่ทำให้คนอยาก deploy เร็ว?

จากประสบการณ์ที่ผ่านมา:

- 🌱 คนที่มีไอเดียดี มักอยากเห็นผลงานตัวเองทำงานเร็ว
- 🌱 กระบวนการปัจจุบันอาจยังไม่ทันกับความรวดเร็วของไอเดีย
- 🌱 เครื่องมือที่มีอยู่อาจยังไม่ตอบโจทย์ self-service

คำถามเปิด: มีประสบการณ์อะไรที่อยากแชร์บ้างคะ?

สิ่งที่เราจะมาเรียนรู้ร่วมกัน

ความท้าทายด้านกระบวนการ

- รอ approval นาน
- Infra อาจยังไม่พร้อมทันที

ความท้าทายด้านเครื่องมือ

- ไม่มี self-service
- ต้องพึ่ง infra team

ความท้าทายด้านข้อมูล

- ขาด documentation ที่ทันสมัย
- ความรู้กระจายอยู่ตามตัวบุคคล

โอกาสในการพัฒนา

- ทำให้ทุกขั้นตอนง่ายและเร็วขึ้น
- สนับสนุนคนที่มีไอเดียดี

สิ่งที่เราอยากสนับสนุน

สิ่งที่ควรมี:

-  ใช้ง่ายเหมือน Vercel/Render
-  Self-service deployment
-  LLM integration + Auto-documentation
-  AI-assisted development

คำถามเปิด: ต้องการอะไรเพิ่มเติม? มีข้อจำกัดอะไร?



Output ของขั้นตอนแรก

รายการสิ่งที่เราอยากสนับสนุน (จัดลำดับความสำคัญ):

- 1.

- 2.

- 3.

- 4.

- 5.

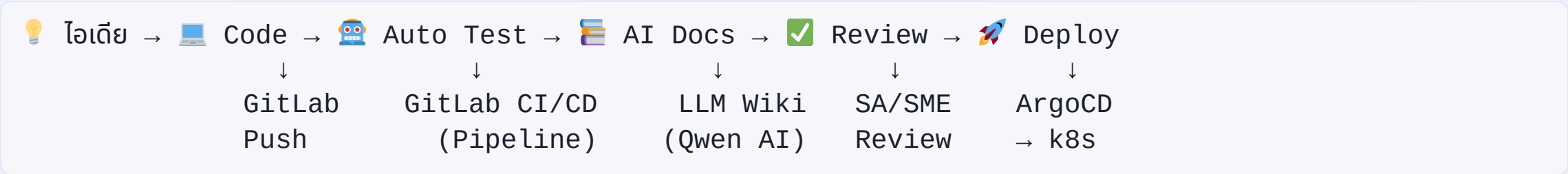


ขั้นตอนที่สอง

Journey: จากไอเดียสู่ระบบจริง

เวลา: 20 นาที | ผู้นำเสนอ: SA

Architecture Overview



สิ่งที่ Framework จะมอบให้ (1/2)

1. Self-Service Deployment

- ใช้งานเหมือน Vercel
- Deploy บน k8s ขององค์กร

2. AI-Powered Documentation

- LLM สร้าง wiki อัตโนมัติ
- อัปเดตเมื่อ code เปลี่ยน

3. Knowledge Management

- ระบบ approve ก่อน publish
- Version control

4. Auto Code Review

- AI ตรวจสอบ code quality, security

สิ่งที่ Framework จะมอบให้ (2/2)

5. MCP Integration

- ทุก app มี MCP server
- AI agent ใช้งานได้ทันที

6. Governance & Support

- สนับสนุนเต็มที่
- ตรวจสอบได้
- Audit trail

Tech Stack (1/3)

Layer	Tool	หน้าที่
Git + CI	GitLab	Git server + CI/CD pipeline
CD	ArgoCD	GitOps deploy to k8s
Orchestration	k8s	Container orchestration



Tech Stack (2/3)

Layer	Tool	หน้าที่
RDB (Primary)	SQL Server	Main database (IT standard)
RDB (Alternative)	PostgreSQL	Open-source option
Vector DB	pgvector	AI embeddings, RAG
DB Operator	CloudNativePG	PostgreSQL automation



Tech Stack (3/3)

Layer	Tool	หน้าที่
LLM Gateway	LiteLLM	Cache, log, route LLM calls
LLM	Alibaba Qwen	Token Plan (credit-based)
Secrets	Vault	API keys, credentials
Registry	Harbor	Container images
Monitoring	Prometheus + Grafana	Metrics, dashboards



Database: Zero Config Provisioning

ปัญหาเดิม:

- ❌ ต้องเปิด ticket ขอ IT สร้าง database (รอ 1-3 วัน)
- ❌ ต้อง config connection string เอง
- ❌ ไม่มี database สำหรับ preview environments
- ❌ ไม่มี vector DB สำหรับ AI/RAG

วิธีแก้:

- ✅ **Self-service database** — สร้างได้ทันทีผ่าน GitLab CI/CD
- ✅ **Auto connection strings** — Inject เป็น environment variables อัตโนมัติ
- ✅ **Database branching** — สร้าง database branch ทุก MR (เหมือน Neon)
- ✅ **Vector DB built-in** — pgvector พร้อมใช้สำหรับ AI

Database Options: เลือกตามความต้องการ

1. SQL Server (IT Standard)

-  IT สนับสนุนเต็มที่
-  Enterprise-grade, HA/DR พร้อม
-  Compatible กับระบบเดิม
-  มี license อยู่แล้ว

เหมาะสำหรับ:

- Legacy systems
- Enterprise applications
- ระบบที่ต้องการ SQL Server features

2. PostgreSQL (Open Source)

-  Free, no license cost
-  pgvector built-in (AI/RAG)
-  CloudNativePG operator
-  Database branching (Neon-style)

เหมาะสำหรับ:

- New applications
- AI/ML projects
- ระบบที่ต้องการ flexibility

3. pgvector (Vector DB)

-  Vector search ใน PostgreSQL
-  ไม่ต้องรัน vector DB แยก
-  Compatible กับ OpenAI embeddings
-  ใช้สำหรับ RAG, semantic search



ตัวอย่าง: สร้าง Database แบบ Zero Config

GitLab CI/CD Template:

```
# .gitlab-ci.yml
include:
  - project: 'platform/db-templates'
    file: '/sqlserver.yml' # หรือ '/postgresql.yml'

variables:
  DB_NAME: myapp
  DB_SIZE: 10Gi
  DB_TYPE: sqlserver # หรือ postgresql
```

ผลลัพธ์:

```
$ git push origin main
# GitLab CI/CD สร้าง database อัตโนมัติ
# Inject connection string เป็น environment variable
🎉 Database ready: myapp-db.apps.company.com
🔑 Connection string injected to: $DATABASE_URL
```

ไม่ต้อง:



Database Branching: เหมือน Neon

NeonDB ทำอย่างไร:

Developer เปิด PR → สร้าง database branch ใน 1 วินาที
→ รัน tests → au branch เมื่อ PR ปิด

วิธีของเรา (ทำเหมือนกัน):

MR #123 → สร้าง database branch: myapp-mr-123
MR #124 → สร้าง database branch: myapp-mr-124
→ รัน tests → au branch เมื่อ MR ปิด

GitLab CI/CD Template:

```
create-db-branch:
  script:
    - |
      # สร้าง database branch (PostgreSQL)
      DB_BRANCH="myapp-mr-${CI_MERGE_REQUEST_IID}"
      kubectl exec postgres-0 -- \
        psql -c "CREATE DATABASE ${DB_BRANCH} WITH TEMPLATE myapp_main;"

      # Inject connection string
      echo "DATABASE_URL=postgresql://..." >> .env
```



Vector DB สำหรับ AI/RAG

ปัญหา:

- AI applications ต้องการ vector storage สำหรับ embeddings
- ต้องรัน vector DB แยก (Pinecone, Weaviate, Milvus)
- ซับซ้อนและแพง

วิธีแก้: pgvector

- ✓ Vector search ใน PostgreSQL เดียวกัน
- ✓ ไม่ต้องรัน vector DB แยก
- ✓ Compatible กับ OpenAI embeddings
- ✓ Free, open-source

ตัวอย่าง:

```
-- Enable pgvector
CREATE EXTENSION vector;

-- Create table with vector column
CREATE TABLE documents (
  id bigserial PRIMARY KEY,
```



Database Lifecycle Management

Auto Backup:

- ✓ Backup อัตโนมัติทุก 6 ชั่วโมง
- ✓ เก็บไว้ 30 วัน
- ✓ Point-in-time recovery
- ✓ เก็บใน S3/MinIO

Auto Migration:

- ✓ Schema migration อัตโนมัติ
- ✓ ใช้ Atlas หรือ Flyway
- ✓ Version control สำหรับ schema
- ✓ Rollback ได้

Auto Cleanup:

- ✓ ลบ database branch เมื่อ MR ปิด
- ✓ ลบ preview databases ที่ไม่ได้ใช้
- ✓ Free up storage อัตโนมัติ

Auto Scaling:

- ✓ Read replicas สำหรับ read-heavy workloads
- ✓ Connection pooling (PgBouncer)
- ✓ Horizontal scaling ได้

Cost Comparison: Database

Feature	NeonDB	Supabase	วิธีของเรา
Cost	\$19/seat/mo	\$25/seat/mo	Free (self-hosted)
SQL Server	✗	✗	✓ มี (IT standard)
PostgreSQL	✓	✓	✓
Vector DB	✓	✓	✓ pgvector
Database branching	✓	✗	✓ มี
Auto backup	✓	✓	✓
Data control	✗ (cloud)	✗ (cloud)	✓ ในองค์กร
Compliance	⚠	⚠	✓ PDPA, ISO

สรุป:

- ✓ ถูกกว่า (Free vs \$19-25/seat/mo)
- ✓ มี SQL Server (Neon/Supabase ไม่มี)
- ✓ Data control + Compliance

Developer Experience: เปรียบเทียบ

NeonDB Experience:

```
$ git push origin main
# Neon สร้าง database branch อัตโนมัติ
# Inject DATABASE_URL
🎉 Database: pr-123.neon.tech
🔑 $DATABASE_URL injected
```

Supabase Experience:

```
$ stripe projects provision supabase
# สร้าง project พร้อม database, auth, storage
🎉 Project ready
🔑 Credentials in .env
```

วิธีของเรา (ใกล้เคียงกัน):

```
$ git push origin main
# GitLab CI/CD สร้าง database อัตโนมัติ
# Inject connection string
🎉 Database: myapp-db.apps.company.com
🔑 $DATABASE_URL injected
```


Database Implementation Plan

Phase 1: SQL Server on Kubernetes (เดือน 1-2)

- [] Deploy SQL Server on k8s (IT standard)
- [] สร้าง GitLab CI/CD template สำหรับ SQL Server
- [] Auto connection string injection
- [] Auto backup (S3/MinIO)

Phase 2: PostgreSQL + pgvector (เดือน 3)

- [] Deploy CloudNativePG operator
- [] Enable pgvector extension
- [] สร้าง GitLab CI/CD template สำหรับ PostgreSQL
- [] Database branching สำหรับ preview environments

Phase 3: Database Lifecycle (เดือน 4-6)

- [] Auto migration (Atlas/Flyway)
- [] Auto cleanup เมื่อ MR ปิด
- [] Read replicas สำหรับ scaling
- [] Documentation + templates

Message Queue: Zero Config Provisioning

ปัญหาเดิม:

- ❌ ต้องเปิด ticket ขอ IT สร้าง RabbitMQ/Kafka cluster (รอ 1-3 วัน)
- ❌ ต้อง config connection string เอง
- ❌ ไม่มี queue สำหรับ preview environments
- ❌ แพง! CloudAMQP \$25-200/mo

วิธีแก้:

- ✅ **Shared cluster** — IT ดูแล cluster เดียว ใช้ร่วมกันทุก app
- ✅ **Auto vhost/topic** — สร้าง resources อัตโนมัติผ่าน Operator
- ✅ **Auto connection strings** — Inject เป็น environment variables อัตโนมัติ
- ✅ **Queue branching** — สร้าง queue branch ทุก MR (เหมือน Neon)
- ✅ **Free** — Self-hosted ถูกกว่า CloudAMQP/Upstash มาก

Message Queue Options: เลือกตามความต้องการ

1. RabbitMQ (แนะนำ — Task Queues)

-  RabbitMQ Cluster Operator (Production-ready)
-  AMQP protocol, routing ยืดหยุ่น
-  เหมาะกับ background jobs, microservices
-  IT คู่ขนาน

เหมาะสำหรับ:

- Task queues (background jobs)
- Microservices communication
- RPC patterns

2. Kafka (High Throughput)

-  Strimzi Operator (CNCF incubating)
-  Event streaming, high throughput
-  เหมาะกับ real-time data, log aggregation
-  Retention policies

เหมาะสำหรับ:

- Event streaming
- Log aggregation
- Real-time analytics

3. Redis (Lightweight)

-  Redis Operator
-  Pub/Sub, caching
-  เหมาะกับ notifications, simple queues



ตัวอย่าง: สร้าง Queue แบบ Zero Config

GitLab CI/CD Template:

```
# .gitlab-ci.yml
include:
  - project: 'platform/queue-templates'
    file: '/rabbitmq.yml' # หรือ '/kafka.yml', '/redis.yml'

variables:
  QUEUE_TYPE: rabbitmq
  VHOST_NAME: myapp
```

ผลลัพธ์:

```
$ git push origin main
# GitLab CI/CD สร้าง vhost + queues อัตโนมัติ
# Inject connection string เป็น environment variable
🎉 Queue ready: myapp-rabbitmq.apps.company.com
🔑 $RABBITMQ_URL injected
```

ไม่ต้อง:

- ❌ เปิด ticket ขอ IT
- ❌ Config connection string เอง

Cost Comparison: Message Queue

Feature	CloudAMQP	Upstash Kafka	วิธีของเรา
Cost	\$25-200/mo	\$10-100/mo	Free (self-hosted)
RabbitMQ	✓	✗	✓
Kafka	✗	✓	✓
Redis	✗	✓	✓
Auto provisioning	✓	✓	✓
Data control	✗ (cloud)	✗ (cloud)	✓ ในองค์กร
Compliance	⚠	⚠	✓ PDPA, ISO

สรุป:

- ✓ ถูกกว่า (Free vs \$25-200/mo)
- ✓ มีทุก options (RabbitMQ, Kafka, Redis)
- ✓ Data control + Compliance
- ✓ ฟีเจอร์ครบ (branching, auto-provisioning)

Message Queue Implementation Plan

Phase 1: RabbitMQ (ເດືອນ 1-2)

- [] Deploy RabbitMQ Cluster Operator
- [] Setup shared RabbitMQ cluster (3 nodes)
- [] ສ້າງ GitLab CI/CD template
- [] Auto vhost provisioning
- [] Auto connection string injection

Phase 2: Kafka (ເດືອນ 3-4)

- [] Deploy Strimzi Kafka Operator
- [] Setup shared Kafka cluster (3 brokers)
- [] ສ້າງ GitLab CI/CD template
- [] Auto topic provisioning
- [] Auto connection string injection

Phase 3: Advanced Features (ເດືອນ 5-6)

- [] Queue monitoring (Prometheus + Grafana)
- [] Auto-scaling based on queue depth



Security & Access Control: 3 Layers of RBAC

คำถามสำคัญ: "Zero Config ปลอดภัยไหม? ทีมอื่นจะ access database/queue ของเราได้ไหม?"

คำตอบ:  ปลอดภัยด้วย 3 Layers ของ RBAC

1. **Kubernetes RBAC** — Namespace isolation (ทีม A เข้าไม่ได้ namespace ทีม B)
2. **Database/Queue RBAC** — Credentials per app (app A access ไม่ได้ DB ของ app B)
3. **Portal RBAC** — UI access control (Developer เห็นเฉพาะ app ของตัวเอง)

Layer 1: Kubernetes RBAC (Namespace Isolation)

หลักการ:

```
k8s Cluster
├── Namespace: team-a
│   ├── App A
│   ├── DB A
│   └── Queue A
├── Namespace: team-b
│   ├── App B
│   ├── DB B
│   └── Queue B
```

สิ่งที่ต้องทำ:

- ✓ Namespace per team/app
- ✓ Role/RoleBinding per namespace
- ✓ NetworkPolicy (ป้องกัน cross-namespace)
- ✓ ResourceQuota (จำกัด CPU/Memory)

ตัวอย่าง NetworkPolicy:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-cross-namespace
  namespace: team-a
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: team-a
```

ผลลัพธ์:

- ✓ ทีม A เข้า namespace ทีม B ไม่ได้
- ✓ Pod ของทีม A communicate กับทีม B ไม่ได้
- ✓ Isolation สมบูรณ์



Layer 2: Database/Queue RBAC (Resource Level)

RabbitMQ:

```

Shared RabbitMQ Cluster
├── VHost: team-a-app-1
│   └── user: team-a-app-1 (access เฉพาะ vhost นี้)
├── VHost: team-b-app-1
│   └── user: team-b-app-1

```

SQL Server:

```

Shared SQL Server
├── Database: team_a_app_1
│   └── login: team_a_app_1 (access เฉพาะ DB นี้)
├── Database: team_b_app_1
│   └── login: team_b_app_1

```

สิ่งที่ต้องทำ:

- ☒ Auto-generate credentials per app
- ☒ Store credentials ใน Vault
- ☒ Inject credentials เป็น environment variables
- ☒ จำกัดสิทธิ์ (app A access ไม่ได้ DB ของ app B)

Workflow:

```

Developer กด "Create Database"
→ Portal สร้าง database + credentials
→ เก็บ credentials ใน Vault
→ Inject เป็น $DATABASE_URL
→ Developer ใช้ได้ทันที
→ ทีมอื่น access ไม่ได้

```

Layer 3: Portal RBAC (User Interface Level)

ใช้ Backstage RBAC Plugin (Open Source)

ตัวอย่าง Roles:

Role	เห็นอะไร	ทำอะไรได้
Developer	App ของตัวเองเท่านั้น	Deploy dev/staging, สร้าง DB/Queue
Tech Lead	App ในทีมตัวเอง	Approve production deploy
SA	ทุก app	Review architecture
Infra	ทุก app + infrastructure	Manage clusters, operators
Admin	ทุกอย่าง	ทุกอย่าง

สิ่งที่ Backstage RBAC ทำได้:

- ✓ ควบคุมการเข้าถึง plugins, routes, data
- ✓ กำหนด roles + permissions ผ่าน UI (no-code)
- ✓ Conditional permissions (dev ทำได้เฉพาะ dev environment)
- ✓ Audit logs (ใครทำอะไร เมื่อไหร่)

ตัวอย่าง: Workflow ที่สมบูรณ์

Developer (team-a) → เข้า Portal



เห็นเฉพาะ App ของ team-a



กด "Create Database"



Portal ตรวจสอบ:

- User เป็น developer ของ team-a? ✓
- team-a มี quota เหลือไหม? ✓
- Database name ซ้ำไหม? ✓



Portal เรียก GitLab CI/CD:

- สร้าง database ใน namespace team-a
- สร้าง credentials ใน Vault
- Inject connection string



Developer ได้ database กันที
(ทีมอื่น access ไม่ได้)

✓ **Zero Config + Secure + Audit ได้!**

Security Implementation Plan

Phase 1 (เดือน 1-2): Kubernetes RBAC

- ☐ Namespace per team
- ☐ Role/RoleBinding per namespace
- ☐ NetworkPolicy (deny cross-namespace)
- ☐ ResourceQuota (CPU/Memory/Storage limits)

Phase 2 (เดือน 3-4): Database/Queue RBAC

- ☐ Auto-generate credentials per app
- ☐ Store credentials ใน Vault
- ☐ Inject credentials เป็น environment variables
- ☐ จำกัดสิทธิ์ per app

Phase 3 (เดือน 5-6): Portal RBAC

- ☐ Setup Backstage (optional)
- ☐ ติดตั้ง RBAC plugin
- ☐ กำหนด roles + permissions
- ☐ Audit logs



Resource Management & Scaling

คำถามสำคัญ: "ต้องเตรียมเครื่อง Server เท่าไหร่? จะ Scale อย่างไร?"

คำตอบ:  ใช้ **Kubernetes Auto-Scaling** + ประมาณการเงินตามจริง

1. **Resource Requirements** — k8s cluster ต้องการเท่าไร?
2. **Scaling Strategy** — Scale Up vs Scale Out
3. **Cost Estimation** — ชื้อ vs เช่า ราคาเท่าไร?

Resource Requirements: k8s Cluster

Minimum Production Cluster:

Control Plane (3 nodes - HA):

- CPU: 4 cores
- RAM: 8 GB
- Storage: 100 GB SSD
- **รวม**: 12 cores, 24 GB RAM

Worker Nodes (เริ่มต้น 3 nodes):

- CPU: 8 cores/node
- RAM: 32 GB/node
- Storage: 500 GB SSD/node
- **รวม**: 24 cores, 96 GB RAM

รวม Cluster:

- **36 cores, 120 GB RAM**
- **1.6 TB Storage**

Resource per App (โดยประมาณ):

App Type	CPU	RAM	Storage
Web App	0.5 core	512 MB	1 GB
API Service	1 core	1 GB	2 GB
Database	2 cores	4 GB	50 GB
Message Queue	1 core	2 GB	10 GB
AI/ML Service	2 cores	4 GB	5 GB

รองรับได้:

- 10-20 apps (mixed workloads)
- 50-100 pods
- 1,000-2,000 requests/second

Scaling Strategy: Up vs Out

Scale Up (Vertical Scaling)

เพิ่มทรัพยากรให้ node เดิม:

- เพิ่ม CPU/RAM ให้ node
- ง่าย ไม่ต้องเพิ่ม node ใหม่
- เหมาะกับ stateful workloads (database)

ข้อดี:

-  ง่าย เร็ว
-  ไม่ต้อง re-architect
-  เหมาะกับ database

ข้อเสีย:

-  มีขีดจำกัด (hardware limit)
-  Downtime ตอน upgrade
-  แพงกว่า (diminishing returns)

Scale Out (Horizontal Scaling)

เพิ่ม node ใหม่เข้า cluster:

- เพิ่ม worker nodes
- k8s กระจาย workload อัตโนมัติ
- เหมาะกับ stateless workloads (web apps)

ข้อดี:

-  ไม่จำกัด (เพิ่มได้เรื่อยๆ)
-  No downtime
-  ถูกกว่า (commodity hardware)

ข้อเสีย:

-  ต้อง design สำหรับ distributed
-  ซับซ้อนกว่า
-  ต้องการ load balancer

คำแนะนำ: ใช้ **Scale Out** เป็นหลัก + **Scale Up**

Kubernetes Auto-Scaling

ใช้ 2 ระบบร่วมกัน:

1. Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```


Cost Estimation: ชื้อ vs เช่า

Option 1: ชื้อเครื่อง (On-Premise)

Hardware (3-year amortization):

Item	Qty	Price/Unit	Total
Dell PowerEdge R750	3	฿350,000	฿1,050,000
(48 cores, 128 GB RAM, 2TB SSD)			
Network Switch	1	฿150,000	฿150,000
(10GbE, 24 ports)			
UPS	1	฿100,000	฿100,000
(10kVA, online)			
รวม Hardware			฿1,300,000

Option 2: เช่าเครื่อง (Cloud/Colocation)

Dedicated Server (3 nodes):

Item	Monthly/Node	Total Monthly
Dell R750	฿45,000	฿135,000
(48 cores, 128 GB RAM, 2TB SSD)		
Network Switch	฿5,000	฿5,000
UPS	฿3,000	฿3,000
รวม	฿53,000	฿143,000

Colocation:

Item	Monthly
Full Rack (42U)	฿42,000

Colocation (Data Center):



คำแนะนำ: Hybrid Approach

Phase 1 (เดือน 1-6): เช่าเครื่อง

- ✓ ไม่ต้องลงทุนสูง
- ✓ ทดสอบระบบก่อน
- ✓ ปรับขนาดได้ตามต้องการ
- **Cost: ~฿200,000/เดือน**

Phase 2 (เดือน 7+): ซื้อเครื่อง

- ✓ ประหยัดกว่าในระยะยาว
- ✓ ควบคุม hardware เอง
- ✓ Depreciation 3-5 ปี
- **Cost: ~฿95,000/เดือน (3-year avg)**

ROI:

- ปีที่ 1: เช่า = ฿2.4M
- ปีที่ 2-3: ซื้อ = ฿1.1M/ปี
- **ประหยัด: ฿1.3M ใน 3 ปี**



Resource Monitoring & Optimization

Monitoring Stack:

1. Prometheus + Grafana

- เก็บ metrics (CPU, RAM, disk, network)
- แสดง dashboard แบบ real-time
- Alert เมื่อ resource ใกล้เคียง

2. Kubernetes Metrics Server

```
kubectl top nodes  
kubectl top pods
```

3. Vertical Pod Autoscaler (VPA)

- แนะนำ resource requests/limits
- ปรับขนาด pods อัตโนมัติ

4. KubeCost

- แสดง cost per namespace/app
- หา optimization opportunities

Optimization Strategies:

1. Right-sizing

- ตั้ง requests/limits ให้เหมาะสม
- ใช้ VPA recommendations
- Avoid over-provisioning

2. Resource Quotas

```
apiVersion: v1  
kind: ResourceQuota  
metadata:  
  name: team-a-quota  
spec:  
  hard:  
    requests.cpu: "10"  
    requests.memory: 20Gi  
    limits.cpu: "20"  
    limits.memory: 40Gi
```

3. Pod Disruption Budgets

ขั้นตอนสาม

ทีมของเรา: ใครช่วยอะไรได้บ้าง

เวลา: 15 นาที | ผู้นำเสนอ: ทุกทีม

ทีมพร้อมสนับสนุน

ทีม	หน้าที่	ช่วยอะไรได้บ้าง
SA	Architecture	ออกแบบระบบ, wiki schema
AI-Eng	LLM Integration	Auto-docs, MCP, AI review
Dev	Implementation	Workflow, templates
Infra	Infrastructure	k8s, monitoring, security

คุณแคมีโอเดีย — ที่เหลือเราช่วย!

LLM Wiki คืออะไร?

หลักการ: ทุก app มี documentation อัตโนมัติ

- AI อ่าน code + design docs → สร้าง structured wiki อัตโนมัติ
- เก็บเป็น markdown files (ไม่ต้องใช้ vector DB)
- อัปเดตอัตโนมัติเมื่อ code เปลี่ยน

Workflow

1. Dev push code → GitLab CI/CD pipeline trigger
↓
2. LLM (Qwen) อ่าน code → สร้าง wiki pages
↓
3. Wiki ถูก commit กลับเข้า repo
↓
4. SA/SME review → approve/reject
↓
5. approve → อนุญาต deploy | reject → แจ้ง Dev แก้ไข

โครงสร้าง LLM Wiki

```
project-wiki/
├── raw/           ← source material (code, design docs)
├── wiki/          ← AI-generated structured knowledge
│   ├── index.md
│   ├── architecture.md
│   ├── api-reference.md
│   ├── deployment-guide.md
│   └── troubleshooting.md
└── outputs/       ← reports, answers
```


KM Approval Process

SME Review

- Subject Matter Expert review ทุก wiki page

Version Control

- Track การเปลี่ยนแปลง, Rollback ได้

Periodic Review

- ตั้งเวลา review ทุก 6 เดือน

Audit Log

- รู้ว่าใคร approve, เมื่อไหร่, อะไรเปลี่ยน

ทำไมต้อง Approve?

เพื่อสร้างคุณภาพ:

-  ข้อมูลถูกต้อง เป็นปัจจุบัน
-  Documentation ที่เชื่อถือได้
-  Quality assurance + Accountability

เพื่อสร้างความเป็นเจ้าของ:

-  ทีมช่วยกันตรวจสอบ
-  ทุกคนมีส่วนร่วมในการสร้างความรู้



ขั้นตอนที่

AI ช่วยอะไรได้บ้าง

เวลา: 15 นาที | ผู้นำเสนอ: AI-Eng

LiteLLM คืออะไร?

LLM Gateway

- รวม LLM calls ทั้งหมดทีเดียว

Caching

- Cache input/output → ลด cost, เพิ่ม speed

Logging

- Log ทุก call → ทำเป็น knowledge base

Rate Limiting

- ควบคุม usage per project



Alibaba Token Plan

កំណែប្រែ Credit ប្រើប្រាស់ per-call

Seat Type	Price	Quota
Standard	\$30/seat/mo	25,000 credits
Pro	\$100/seat/mo	100,000 credits
Max	\$200/seat/mo	250,000 credits

Models: Qwen, DeepSeek, Kimi, GLM, MiniMax



Alibaba Token Plan — Models

Brand	Models
Qwen	qwen3.7-max, qwen3.7-plus, qwen3.6-plus, qwen3.6-flash
DeepSeek	deepseek-v4-pro, deepseek-v4-flash, deepseek-v3.2
Kimi	kimi-k2.7-code, kimi-k2.6, kimi-k2.5
GLM	glm-5.2, glm-5.1, glm-5
MiniMax	MiniMax-M2.5

Architecture

User Request → 9router (round-robin)



LiteLLM Proxy



API Key 1

(Seat 1)

API Key 2

(Seat 2)

Alibaba Qwen API

ประโยชน์: กระจาย load, หลีกเลียง rate limits, serve 10+ users

Knowledge Base จาก LLM Logs

Input/Output ที่ log ไว้ใน LiteLLM:

- นำมาสร้าง wiki ได้
- Cache hits → ลด cost, เพิ่ม speed

Workflow:

LLM Call → LiteLLM Log → Langfuse → Analyze → Wiki Update

ขั้นตอนห้า

เริ่มต้นอย่างไร

เวลา: 15 นาที | ผู้นำเสนอ: ทุกทีม

3 ขั้นตอนง่ายๆ

1. แอร์โฮเตีย

- บอกเราว่าอยากทำอะไร
- ไม่ต้องมี code ก็ได้

2. ทีมช่วย setup

- เราเตรียม infra ให้
- AI ช่วยสร้างเอกสาร

3. Deploy ได้เลย

- ใช้ง่ายเหมือน Vercel
- แต่ปลอดภัย + มี backup + มี monitoring

Network & Access: ไม่ต้องกังวลเรื่อง Port และ DNS

ปัญหาเดิม:

- ❌ User ต้องรู้ว่า app รับบน port ไหน
- ❌ ต้องเปิด ticket ขอ IT สร้าง DNS record (รอ 1-3 วัน)
- ❌ IT เป็น bottleneck สร้าง DNS ให้ทุก app

วิธีแก้:

- ✅ **Wildcard DNS** — IT ทำครั้งเดียว `*.apps.company.com`
- ✅ **Ingress Controller** — Traefik จัดการ routing อัตโนมัติ
- ✅ ไม่ต้องเปิด ticket อีกเลย!

Wildcard DNS: IT ทำครั้งเดียว จบ!

สิ่งที่ IT ทำ (ครั้งเดียว):

1. สร้าง DNS: `*.apps.company.com`
2. ชี้ไป IP ของ k8s cluster
3. ตั้งค่า TLS certificate (Let's Encrypt)

เสร็จแล้ว! ไม่ต้องทำอะไรอีก

สิ่งที่ User ทำ:

```
# แค่งำหนดชื่อใน Ingress
spec:
  rules:
    - host: my-app.apps.company.com
```

ผลลัพธ์:

-  App เข้าถึงได้ทันที
-  ไม่ต้องรอ IT
-  ไม่ต้องรู้ port

ตัวอย่าง: Deploy แล้วได้ URL กันที

```
# User push code → GitLab CI/CD deploy
$ git push origin main

# GitLab CI/CD สร้าง Ingress อัตโนมัติ
$ kubectl apply -f ingress.yaml

# ✅ App พร้อมใช้งาน!
🎉 Your app is live at: http://my-app.apps.company.com
```

ไม่ต้อง:

- ❌ เปิด ticket ขอ DNS
- ❌ รอ IT สร้าง record
- ❌ รู้ว่า app รับบน port ไหน
- ❌ ตั้งค่า firewall

แค่ push code → ได้ URL กันที!

VS เปรียบเทียบกับ Vercel/Cloudflare

Vercel/Cloudflare ทำอย่างไร?

- ✓ Push code → ได้ URL อัตโนมัติ
- ✓ Preview ทุก PR อัตโนมัติ
- ✓ Zero config (ไม่ต้องเขียน YAML)
- ✓ Global CDN

ข้อเสีย:

- ✗ ข้อมูลอยู่บน cloud ของเขา
- ✗ ค่าใช้จ่าย \$20/seat/month
- ✗ ควบคุมไม่ได้ (PDPA, ISO)

วิธีของเรา (k8s + GitLab)

- ✓ Push code → ได้ URL อัตโนมัติ (ผ่าน GitLab CI/CD)
- ✓ Preview ทุก MR อัตโนมัติ
- ✓ Zero config (แค่ include template)
- ✓ ข้อมูลอยู่ในองค์กร

ข้อดี:

- ✓ **Free** (ประหยัด \$20/seat/month)
- ✓ **Data control** (PDPA, ISO)
- ✓ **Compliance** (audit ได้)

UX ต้องใกล้เคียง Vercel

Vercel experience:

```
$ git push origin main  
# 30 วินาที...  
🎉 https://my-app.vercel.app
```

วิธีของเรา (ทำให้ใกล้เคียง):

```
$ git push origin main  
# GitLab CI/CD ทำทุกอย่างอัตโนมัติ  
# 1-2 นาที...  
🎉 https://my-app.apps.company.com
```

สิ่งที่ User ต้องทำ:

```
# .gitlab-ci.yml (แค่ include template)  
include:  
  - project: 'platform/ci-templates'  
    file: '/k8s-deploy.yml'  
  
variables:  
  APP_NAME: my-app  
  APP_PORT: 3000
```



Preview Deployment (เหมือน Vercel)

Vercel:

```
MR #123 → https://my-app-git-123.vercel.app
MR #124 → https://my-app-git-124.vercel.app
```

วิธีของเรา:

```
MR #123 → https://my-app-mr-123.apps.company.com
MR #124 → https://my-app-mr-124.apps.company.com
```

GitLab CI/CD template สร้าง preview อัตโนมัติ:

```
deploy:preview:
  script:
    - |
      kubectl apply -f - <<EOF
      apiVersion: networking.k8s.io/v1
      kind: Ingress
      metadata:
        name: $CI_PROJECT_NAME-mr-$CI_MERGE_REQUEST_IID
      spec:
        rules:
          - host: $CI_PROJECT_NAME-mr-$CI_MERGE_REQUEST_IID.apps.company.com
            http:
```


ทำไม User จะยอมเปลี่ยน?

Incentive ที่ชัดเจน:

1. Cost

- Vercel: \$20/seat/month
- เรา: **Free**
- ประหยัด \$240/year/คน!

2. Data Control

- Vercel: ข้อมูลอยู่บน cloud
- เรา: อยู่ในองค์กร
- PDPA, ISO compliant

3. Compliance

- Vercel: ต้องตรวจสอบ
- เรา: ควบคุมเองได้
- Audit ได้ทุกขั้นตอน

4. Custom Backend

- Vercel: จำกัด
- เรา: อะไรก็ได้
- Database, API, etc.

สรุป:

-  ประหยัดเงิน
-  ควบคุมข้อมูล
-  Compliance ดี
-  ยืดหยุ่นกว่า

 ใช้ Open Source หนึ่งชุด

Phase	สิ่งที่ต้องทำ	Open Source Tool	Cost
Phase 1: UX	Zero config deployment	GitLab CE	Free
Phase 2: Incentive	Documentation	ไม่ต้องใช้ tool	Free
Phase 3: Migration	Templates + guides	ไม่ต้องใช้ tool	Free
Phase 3 (Advanced)	Self-Service Portal	Backstage (Spotify)	Free

รวม Cost: \$0 (Free หนึ่งชุด!)

3 Phases Implementation

Phase 1: ทำให้ UX ใกล้เคียง Vercel (เดือน 1-2)

- [] GitLab CI/CD template (Zero config)
- [] Preview deployment อัตโนมัติทุก MR
- [] Auto cleanup เมื่อ merge/close MR
- [] Documentation สั้นๆ (1 หน้า)

Phase 2: เพิ่ม Incentive (เดือน 3)

- [] Cost comparison (Vercel \$20/seat vs เสร Free)
- [] Compliance benefits (PDPA, ISO)
- [] Data control benefits
- [] Success stories จาก pilot projects

Phase 3: Migration ง่าย (เดือน 4-6)

- [] Migration guide (15-30 นาที)
- [] Template สำเร็จรูป (5-10 templates)
- [] Documentation ชัดเจน
- [] (Optional) Backstage Self-Service Portal



Phase 1: Foundation (Month 1-2)

Infrastructure Setup:

- [] Setup k8s cluster (3 nodes)
- [] Setup GitLab + GitLab Runner
- [] Deploy ArgoCD, Harbor, Traefik
- [] Deploy LiteLLM + Redis + Langfuse

Deliverables: k8s cluster + GitLab repo + LLM gateway



Phase 2: Automation (Month 3-4)

AI Integration:

- [] Integrate Alibaba Qwen API
- [] Build auto-doc generation pipeline
- [] Implement KM approval workflow
- [] Create MCP server template

Deliverables: Auto-doc + KM workflow + MCP template



Phase 3: Polish (Month 5-6)

User Experience:

- [] Build self-service portal
- [] Implement governance policies
- [] Onboard 3 pilot projects
- [] Training & documentation

Deliverables: Portal + 3 pilots + Training materials

MCP Server Auto-Registration

դի օրն MCP server

```
from fastmcp import FastMCP
mcp = FastMCP("my-app")

@mcp.tool()
def query_data(query: str) -> dict:
    """Query data from app"""
    pass
```

Auto-register to MCP catalog on deploy



Auto MR & Code Review

```
stages:
  - review

ai-review:
  stage: review
  image: python:3.11
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
  script:
    - pip install openai
    - python scripts/ai-review.py
      --llm_endpoint "http://litellm:4000/v1"
      --model "qwen-plus"
      --mr_iid $CI_MERGE_REQUEST_IID
```


Immediate Actions (อัปเดตหน้า)

Infra&Security:

- Setup k8s cluster (test)
- Deploy LiteLLM + Redis

AI-Eng:

- ทดสอบ Alibaba Qwen API
- สร้าง prototype auto-doc

SA:

- ออกแบบ wiki schema
- กำหนด KM approval criteria

Dev:

- Setup GitLab CI/CD pipelines
- สร้าง project template

Pilot Projects

เลือก 3 projects จาก Maintenance (User)

Criteria:

- Complexity ปานกลาง
- Team พร้อมร่วมมือ
- มี business impact ชัดเจน

Timeline: Onboard 2 สัปดาห์ → Deploy แรกภายใน 1 เดือน



Success Metrics

Developer Satisfaction

- Target: >80% satisfaction

Deployment Time

- Target: <15 นาที

Documentation Coverage

- Target: 100% มี wiki

Self-driven Solutions on Internal Platform

- Target: เพิ่ม 50% ใน 3 เดือน



Summary

จุดเริ่มต้น:

- ไอเดียที่ดีสมควรได้รับการสนับสนุน
- ทำให้ทุกขั้นตอนง่ายและเร็วขึ้น

แนวทาง:

- AI-Powered Framework
- LLM Wiki + KM Approval

Tech Stack:

- k8s + ArgoCD + GitLab
- LiteLLM + Alibaba Qwen

Timeline: 6 months, 3 phases



ขอบคุณค่ะ

ทุกไอเดียสมควรได้รับการสนับสนุน

Project: `~/projects/ai-powered-internal-developer-framework`

Appendix: Backup Data

ข้อมูลสำรอง (ไม่พูดในการประชุม):

1. Alibaba Token Plan Details

- Full model list, credit calculation, pricing
- ดูที่: `docs/backup/alibaba-token-plan.md`

2. CI/CD Comparison

- GitLab CI vs GitHub Actions vs Argo Workflows
- ดูที่: `docs/backup/cicd-comparison.md`